

Kontrollstrukturen

Python vs. Java – am Beispiel eines Notenrechners

Der Code

PYTHON · notenrechner.py

```
# Notenrechner: Noten eingeben, Durchschnitt berechnen und bewerten
# (österreichisches Notensystem 1-5)

noten = []

print("Gib deine Noten ein (1-5). Mit 'fertig' beenden.")

while True:
    eingabe = input("Note: ").strip().lower()

    if eingabe == "":
        pass
    elif eingabe == "fertig":
        break
    else:
        try:
            note = int(eingabe)
            if 1 <= note <= 5:
                noten.append(note)
            else:
                print("Nur Noten von 1 bis 5 sind erlaubt.")
        except ValueError:
            print("Das ist keine Zahl. Bitte nochmal.")

if len(noten) == 0:
    print("Keine Noten eingegeben.")
else:
    summe = 0
    for n in noten:
        summe += n
    schnitt = summe / len(noten)

    print(f"\nAnzahl Noten: {len(noten)}")
    print(f"Durchschnitt: {schnitt:.2f}")

    if 5 in noten:
        print("Achtung: Mindestens ein 'Nicht genügend' dabei!")
    elif schnitt <= 1.5:
        print("Sehr stark, weiter so!")
    elif schnitt <= 2.5:
        print("Solides Ergebnis.")
    else:
        print("Da geht noch was.")
```

lila = Schlüsselwort **rot** = Text (String) **grün** = Kommentar

Auf einen Blick

Struktur	Python	Java	Kernunterschied
Blöcke	: + Einrückung	{ }	Einrückung ist in Python Pflicht
if	if / elif / else	if / else if / else	elif statt else if, keine Klammern
while	while True:	while (true) { }	Python hat kein do-while
for	for n in liste:	for (int n : liste) for (i=0; i<n; i++)	Python hat nur die foreach-Form
break	break	break;	gleich (Java zusätzlich mit Label)
pass	pass	{ } (leer)	gibt es in Java nicht
Fehler	try / except	try / catch	except statt catch, andere Fehlernamen

1 Blöcke: Klammern vs. Einrückung

JAVA

```
if (note > 3) {  
    System.out.println("x");  
}
```

PYTHON

```
if note > 3:  
    print("x")
```

Merke

- Python: **Doppelpunkt** öffnet den Block, **Einrückung** bestimmt, was dazugehört.
- Java: **geschweifte Klammern**, Einrückung ist nur Optik.
- Falsche Einrückung = in Python Fehler oder andere Logik.

2 if / elif / else

JAVA

```
if (schnitt <= 1.5) {  
    ...  
} else if (schnitt <= 2.5) {  
    ...  
} else {  
    ...  
}
```

PYTHON

```
if schnitt <= 1.5:  
    ...  
elif schnitt <= 2.5:  
    ...  
else:  
    ...
```

Merke

- **elif** statt **else if**, Bedingung ohne runde Klammern.
- Python akzeptiert jeden Wert als Bedingung: **if noten:** = „Liste nicht leer“. Java braucht ein boolean.
- Kurzform: Java **c ? a : b** → Python **a if c else b**
- Statt **switch** gibt es in Python **match / case** (ab Version 3.10).

3 while-Schleife

JAVA

```
while (true) {  
    ...  
}
```

PYTHON

```
while True:  
    ...
```

Merke

- **True** / **False** in Python großgeschrieben.
- Python hat **kein do-while**. Ersatz: **while True:** + **break**.

4 for-Schleife

JAVA

```
for (int i = 0; i < 5; i++) { }  
for (int n : noten) { }
```

PYTHON

```
for n in noten:  
    ...
```

Ziel	Java	Python
0 bis 4 zählen	<code>for (int i=0; i<5; i++)</code>	<code>for i in range(5):</code>
1 bis 10, Zweierschritte	<code>for (int i=1; i<=10; i+=2)</code>	<code>for i in range(1, 11, 2):</code>
Index + Element	<code>noten.get(i)</code>	<code>for i, n in enumerate(noten):</code>

Merke

- Python hat **nur eine** for-Schleife, und die ist ein **foreach**.
- Zählen geht über **range()**, Index über **enumerate()**.
- Die Schleifenvariable existiert in Python auch nach der Schleife weiter.

5 break (und continue)

JAVA

```
for (int n : noten) {  
    if (n == 5) break;  
}
```

PYTHON

```
for n in noten:  
    if n == 5:  
        break  
else:  
    print("Alles positiv")
```

Merke

- **break** und **continue** funktionieren gleich.
- Nur Java: **break aussen**; verlässt mehrere Schleifen auf einmal (Label).
- Nur Python: **else** nach einer Schleife läuft, wenn **kein break** kam.

6 pass

JAVA

```
if (eingabe.isEmpty()) {  
    // einfach leer lassen  
}
```

PYTHON

```
if eingabe == "":  
    pass
```

Merke

- `pass` = „tu nichts“. Nötig, weil ein Python-Block nicht leer sein darf.
- Java braucht das nicht, ein leerer Block `{ }` reicht.
- Auch als Platzhalter: `def funktion(): pass`

7 try / except

JAVA

```
try {  
    int note = Integer.parseInt(x);  
} catch (NumberFormatException e) {  
    ...  
} finally {  
    ...  
}
```

PYTHON

```
try:  
    note = int(x)  
except ValueError:  
    ...  
finally:  
    ...
```

Merke

- `except` statt `catch`, Fehlervariable optional: `except ValueError as e:`
- Andere Fehlernamen: `ValueError` (Python) ↔ `NumberFormatException` (Java).
- Mehrere Fehler: Java `catch (A | B e)` → Python `except (A, B):`
- Python hat zusätzlich `else` (läuft, wenn kein Fehler kam). `finally` gibt es in beiden.
- Java erzwingt bei „checked exceptions“ eine Behandlung. Python nicht.

Zusammenfassung

Die 4 wichtigsten Unterschiede

- **Einrückung statt Klammern** – in Python ist das Layout Teil der Syntax.
- **Weniger Varianten** – kein do-while, nur foreach-for, kein switch (sondern match).
- **Python-Extras** – `pass`, `elif`, `else` bei Schleifen und try.
- **Java ist strenger** – Typen, Klammern und Fehlerbehandlung werden vom Compiler geprüft.